Mehran University of Engineering & Technology, Jamshoro Sindh Pakistan

Complex Engineering Problem Report:
Operating Systems

SUBJECT TEACHER:
Engr. Zoha Memon

SUBMITTED BY:
24CS070, 24CS028, 24CS062

DEPARTMENT OF COMPUTER SYSTEMS ENGINEERING

Department of Computer Systems Engineering Mehran University of Engineering and Technology, Jamshoro			
Course: Operating Systems (CS-261)			
Instructor	Engr. Zoha Memon	**Assignment Type**	Complex Engineering Problem
Semester	4th	**Year**	2nd
Submission Deadline	24-04-2026	**Assessment Score**	10

Complex Engineering Problem - Characteristics		
1	Depth of knowledge Required	☑
2	Range of Conflicting Requirements	☑
3	Depth of Analysis Required	☑
4	Infrequently Encountered Issues Involved	☐
5	Beyond codes/standards of practice	☑
6	Diverse groups of stakeholders with widely varying needs involved	☐
7	Interdependence (high level problems including many component parts/sub-problems)	☑
8	Have significant consequences in a range of contexts	☐
9	Judgement (Require judgement in decision making)	☑

Problem Description

Project Objectives:

Kernel Development: Design and implement a basic kernel for the operating system that manages resources and provides essential services like process management, memory management, and file system access.

Process Scheduling: Implement a basic Round Robin scheduling algorithm that allocates equal CPU time to each process in a fixed order, ensuring simple and fair execution of running processes.

Memory Management: Develop a basic memory management system that handles simple allocation and deallocation of memory for running processes, keeping track of used and free memory blocks.

File System: Design a hierarchical file system that allows users to store, retrieve, and manage data effectively with support for various file types and permissions.

Device Drivers: Implement device drivers to enable communication between the OS and hardware peripherals, such as graphics cards, network interfaces, and input/output devices.

Inter-process Communication (IPC): Create mechanisms for interprocess communication, facilitating seamless data sharing and collaboration among different processes.

Security Features: Integrate security measures like user authentication, access control, and encryption to safeguard the OS and user data from unauthorized access.

Graphical User Interface (GUI): Develop an intuitive GUI that provides users with a visually appealing

and user-friendly interface to interact with the operating system.

Error Handling and Logging: Implement a comprehensive error handling and logging system to identify, report, and address system failures and anomalies.

Performance Optimization: Monitor basic system performance metrics such as CPU usage and memory consumption, and apply simple techniques to avoid unnecessary overhead during process execution.

Project Deliverables:

Technical Report: A comprehensive technical report detailing the OS design, architecture, and implementation, including documentation of key algorithms and methodologies.

Source Code: Complete source code of the operating system, well-organized and thoroughly commented for ease of understanding.

User Documentation: User guides and manuals explaining the system's functionalities, installation, and usage instructions.

Table of Contents

1. Project Overview
2. System Architecture
3. Key Algorithms and Methodologies
4. Implementation
5. GUI Tabs
6. User Documentation
7. How to Run
8. Project Structure
9. Shell Commands Reference (Kafka Shell)
10. Conclusion

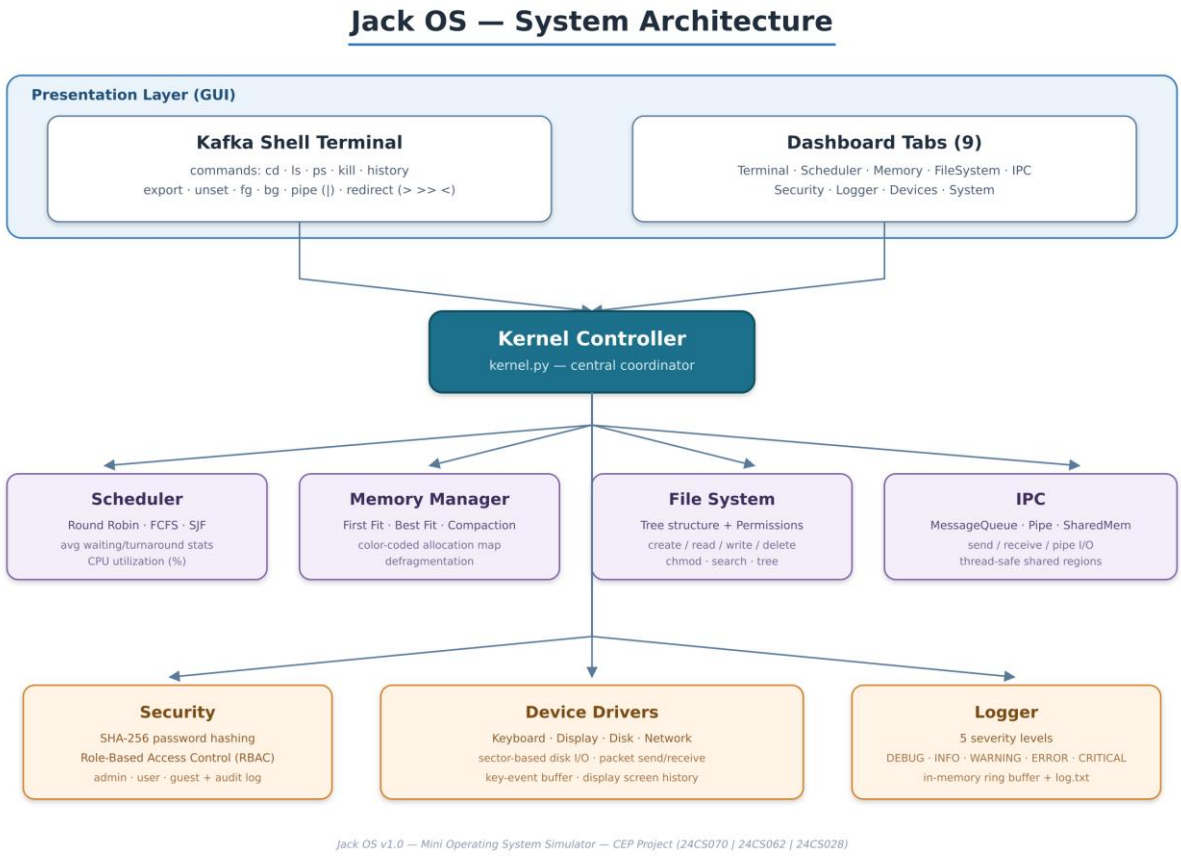
Jack OS — Mini Operating System

Technical Report: OS Design, Architecture, and Implementation

1. Overview

This technical report provides a comprehensive overview of the design, architecture, and implementation of Jack OS — a Mini Operating System Simulator featuring the kafka Shell, a lightweight custom command-line interface. The system is developed entirely in Python and simulates core OS functionalities including process scheduling, memory management, file system operations, inter-process communication (IPC), device driver simulation, security and authentication, and centralized logging. The graphical interface is built with Tkinter and provides nine interactive tabs alongside the kafka Shell terminal for command-line interaction

2. Design and Architecture



The architecture of Jack OS is modular, with a central Kernel Controller coordinating all OS subsystems. The two-layer design consists of a Presentation Layer (GUI + Kafka Shell) and a Core Modules Layer.

System Architecture Layers

- **Presentation Layer (GUI):** A Tkinter-based dashboard with 9 tabs — Terminal (Kafka Shell), Scheduler, Memory, FileSystem, IPC, Security, Logger, Devices, and System overview.

- **Kernel Controller (kernel.py):** The central coordinator that initializes all subsystems on boot and provides spawn_process(), kill_process(), and system_status() methods.

- **Core Modules:** Seven independent modules — Scheduler, Memory Manager, File System, IPC, Security, Device Drivers, and Logger — each with well-defined interfaces

Module Summaries

Process Scheduler (modules/scheduler.py)

Implements three scheduling algorithms: Round Robin (preemptive, configurable time quantum), FCFS (First Come First Served), and SJF (Shortest Job First). The Process class tracks pid, name, burst, original_burst, priority, waiting, turnaround, arrival, and finish_time. After each run the scheduler reports average waiting time, average turnaround time, total time, and CPU utilization percentage.

Memory Manager (modules/memory.py)

Simulates contiguous memory allocation using First Fit (allocates the first sufficiently large free block) and Best Fit (allocates the smallest sufficient block, minimizing wasted space). The Compaction operation defragments memory by moving all allocated blocks to the front. The GUI provides a real-time color-coded visual memory map per process.

File System (modules/filesystem.py)

A tree-structured virtual file system using Node objects. Each node stores name, is_file flag, content, children, owner, permissions (Unix-style strings such as rw-r--r--), creation timestamp, modification timestamp, and size. Supported operations: create(), create_dir(), write() (with permission check), read() (with permission check), delete(), list_dir(), chmod(), move(), search() (recursive by name), and tree() (full tree display).

Inter-Process Communication — IPC (modules/ipc.py)

Three IPC mechanisms are implemented: (1) Message Queue — sender/receiver labeled messages with full history; (2) Pipe — a named unidirectional buffered data channel between processes; (3) Shared Memory — a thread-safe key-value region protected by threading.Lock. The IPC Manager class provides a unified interface for all three mechanisms.

Security Module (modules/security.py)

Provides user authentication with SHA-256 password hashing, session token generation (16-character hex token), and role-based access control (RBAC) with three roles: admin (full access), user (read/write/scheduling), and guest (read-only). Accounts are locked after three consecutive failed login attempts. A comprehensive audit log records all security events with timestamps.

Device Drivers (modules/driver.py)

Four simulated I/O devices: KeyboardDevice (key event buffer and event history), DisplayDevice (timestamped output buffer and screen history), DiskDevice (sector-based read/write with I/O counters), and NetworkDevice (connect/disconnect and packet send/receive simulation). All devices are managed through the DeviceDriver manager class.

Logger (modules/logger.py)

A centralized multi-level logging system with five severity levels: DEBUG, INFO, WARNING, ERROR, and CRITICAL. Each log entry records timestamp, level, module name, and message. The logger maintains an

in-memory ring buffer (500 entries), persists to log.txt, and supports filtered retrieval by level and module name.

Kafka Shell Terminal (gui/terminal.py)

A fully functional command-line interface embedded in the GUI Terminal tab. It replicates all features of the C-based Isekai Shell (functions.c) including: command parsing and dispatch, I/O redirection (>, >>, <), command piping (|), background job execution (&), job control (fg, bg, jobs), environment variables (export, unset, env), command history with Up/Down arrow navigation, and Tab key auto-completion.

3. Key Algorithms and Methodologies

Command Parsing (Kafka Shell)

- Input line is scanned for special tokens: | (pipe), >, >>, < (redirection), & (background). If found, the appropriate handler is invoked before general tokenization.
- Environment variable substitution ($VAR) is performed on the full input line before tokenization.
- The first token is looked up in the COMMANDS registry dictionary. The matching handler is called with remaining tokens as arguments.

Process Scheduling Algorithms

- **Round Robin:** Processes are placed in a deque. Each iteration, the front process runs for min(quantum, remaining_burst) time units, then re-added to the back if burst remains. Waiting time recorded at first execution. Turnaround = finish_time - arrival_time.
- **FCFS**: Processes sorted by arrival time, each runs to completion. Simple but may cause the convoy effect for short processes behind long ones.
- **SJF:** Processes sorted by burst time (shortest first). Provably optimal average waiting time for non-preemptive scheduling.

Memory Allocation Algorithms

- First Fit: Scans the memory array from position 0, allocates the first contiguous free block of sufficient size. Bug in original code fixed: range(len-size) was replaced with while i <= total_size - size to avoid missing the last valid slot.
- Best Fit: Scans all free contiguous blocks and selects the one that leaves the smallest leftover fragment, reducing external fragmentation compared to First Fit.
- Compaction: All occupied cells are collected, written from position 0, and the allocation map is rebuilt. Eliminates all external fragmentation in a single pass.

Forking and Job Control Simulation

- Background jobs (&) are spawned in Python daemon threads, recorded in the jobs list with id, command string, and status (running/stopped).
- fg <id> looks up the job record and sets status to running (simulating waitpid). bg <id> similarly marks a stopped job as running (simulating SIGCONT).
- The shell displays [job_id] at background launch, matching POSIX shell behavior from the C implementation.

Signal Handling

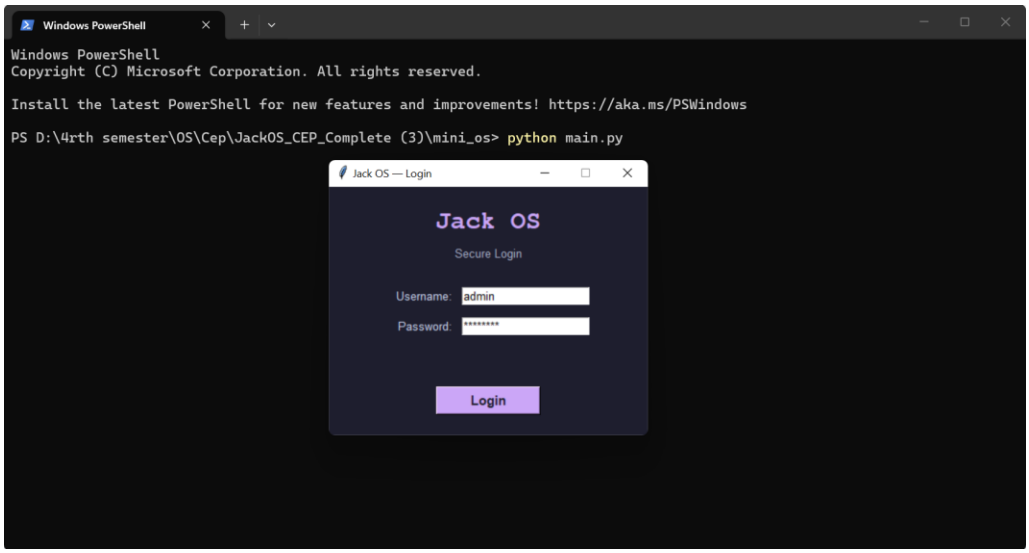
- The Kafka Shell handles errors gracefully — unknown commands print a "command not found" message rather than crashing, mirroring the SIGINT handler in the C shell.

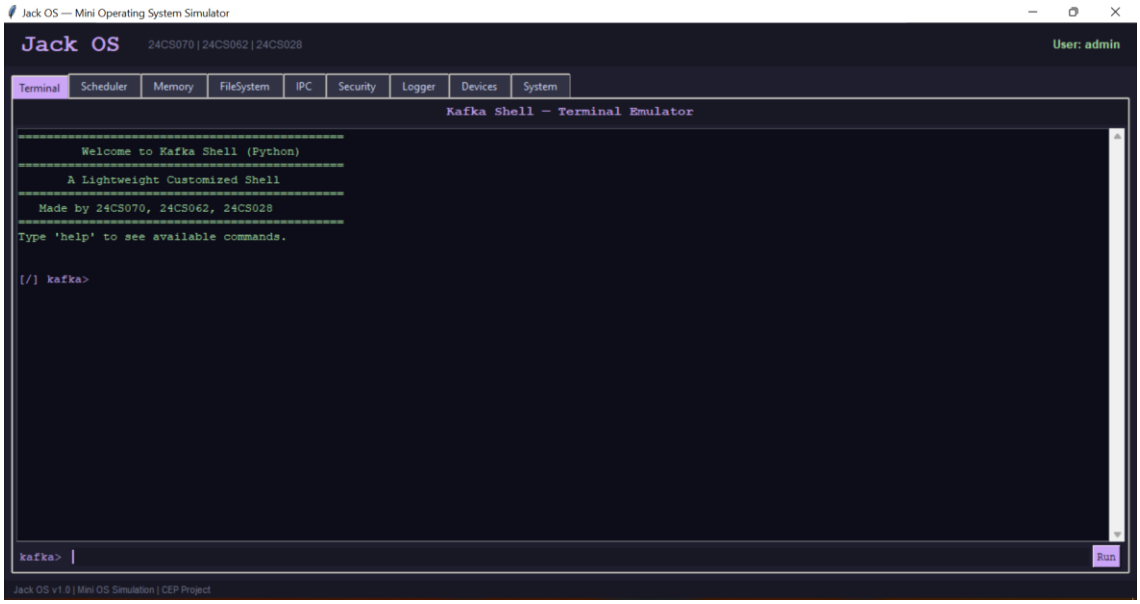
- The security module simulates account locking (analogous to fail2ban) after 3 failed authentication attempts.

4. Implementation

User Interface — Login Screen

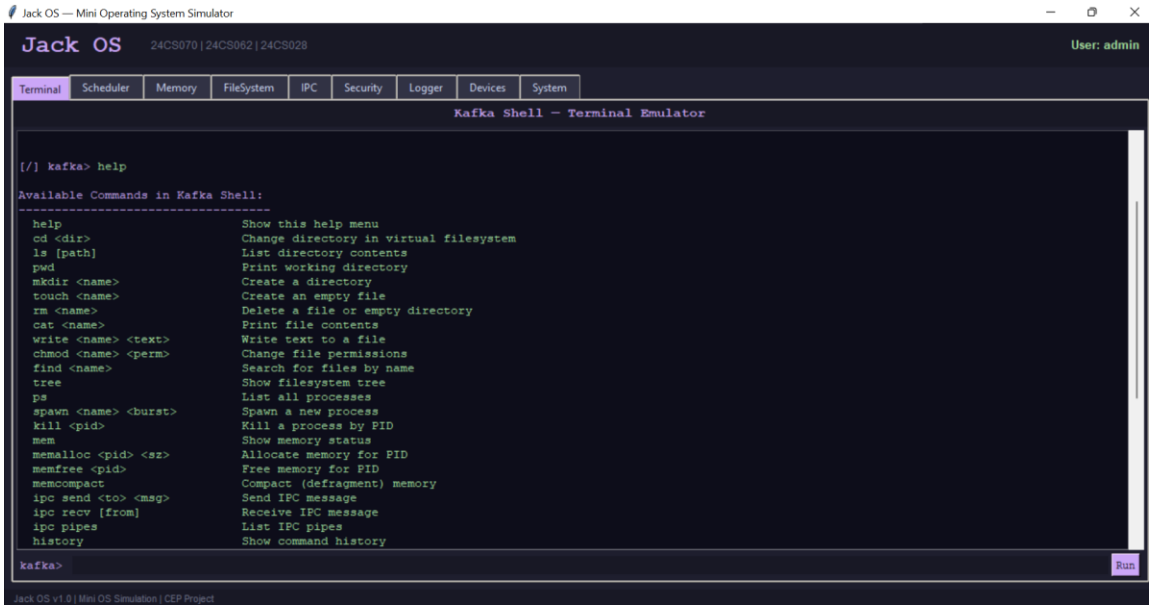
On launch, the application presents a login window before the main dashboard. The user must enter valid credentials. Default accounts: admin / admin123 (full access) and guest / guest (read-only). After three failed attempts, the account is locked and an admin must unlock it.





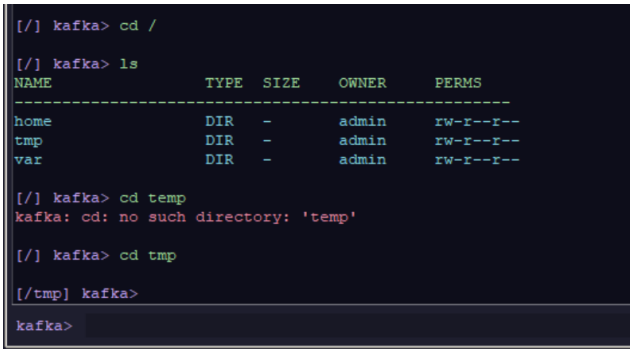
Help Command

The help command in the Kafka Shell displays all available commands with their syntax and a one-line description. The prompt is Kafka> throughout the shell.



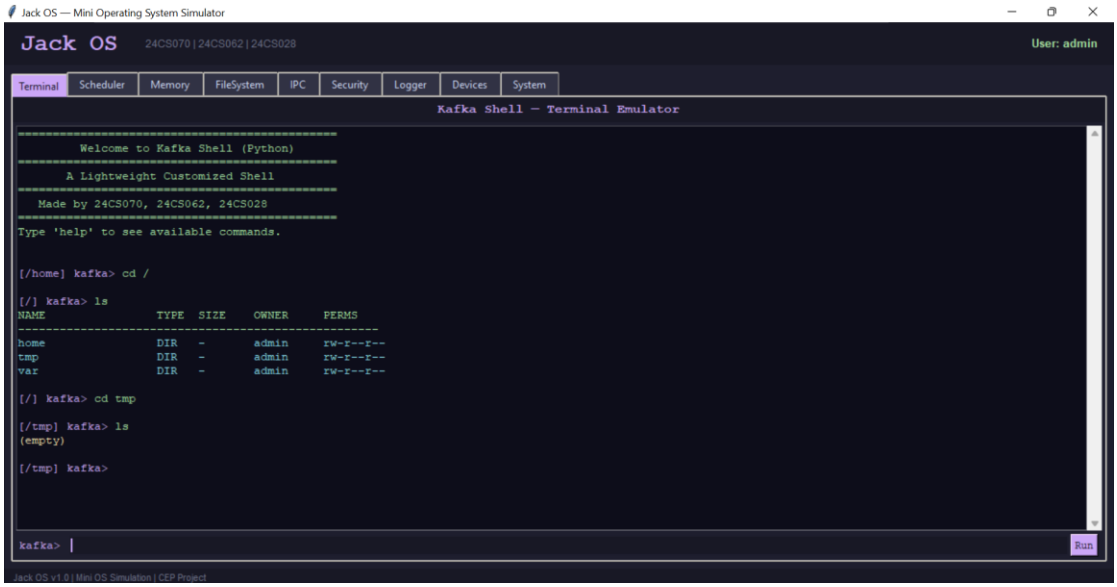
CD Command

Changes the current working directory in the virtual file system. Supported forms: cd <dir> (enter subdirectory), cd .. (go up one level), cd / (go to root). The current path is shown in the prompt.



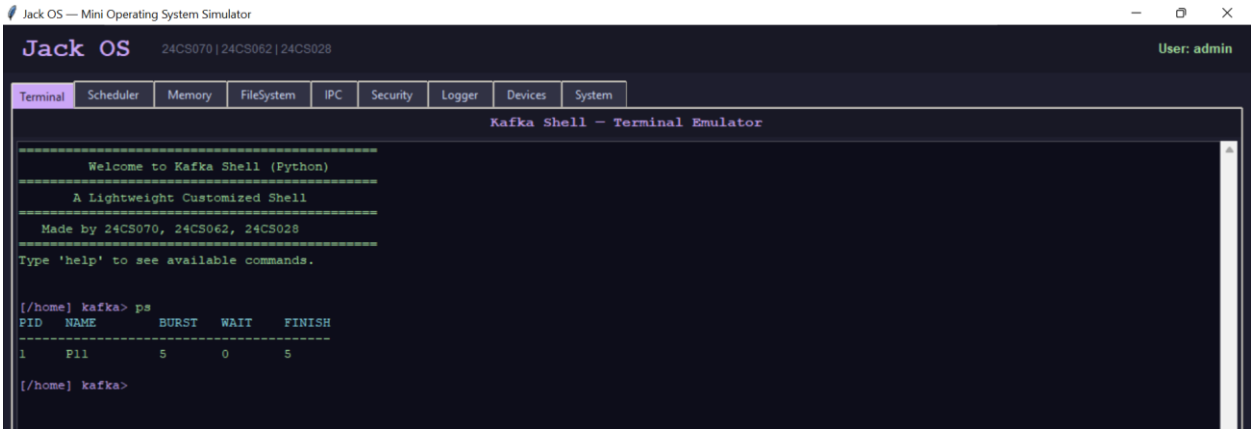
ls Command

Lists contents of the current directory. Each entry shows: name, type (FILE or DIR), size in bytes, owner, and Unix-style permissions string (e.g., rw-r--r--).



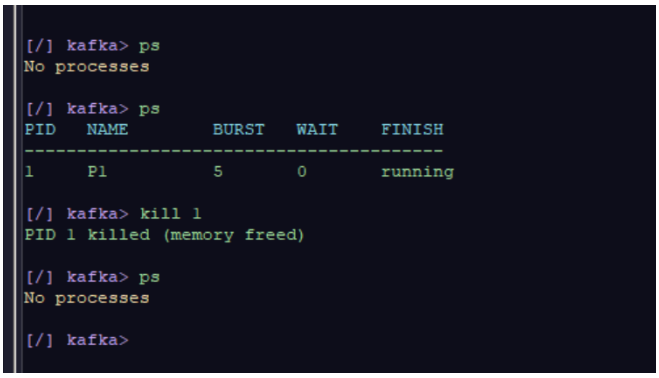
ps Command

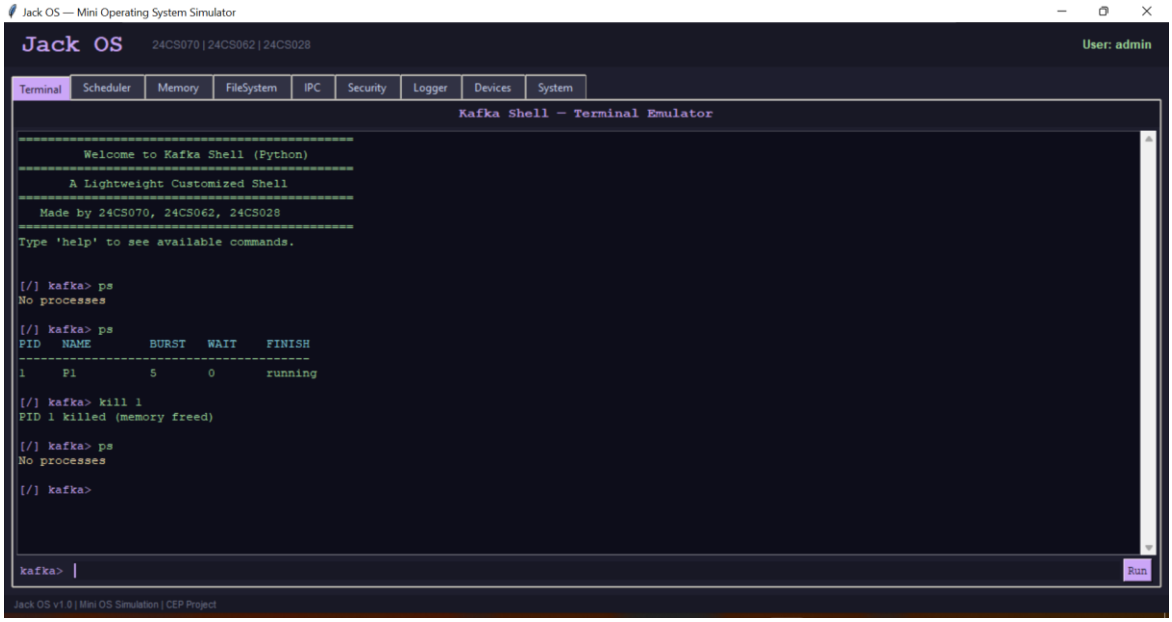
Lists all processes currently registered in the scheduler. Columns: PID, name, burst time, waiting time, and finish time. Running processes show "running" in the finish column.



kill Command

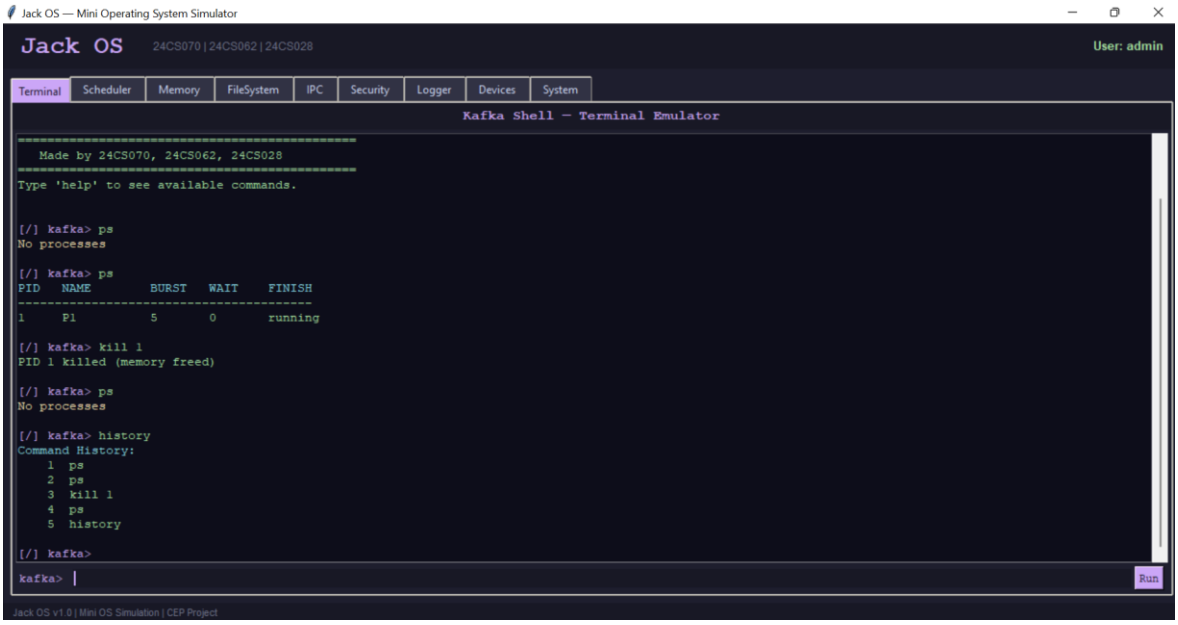
Terminates a process by its PID and simultaneously releases all memory allocated to that process. Usage: kill <pid>.





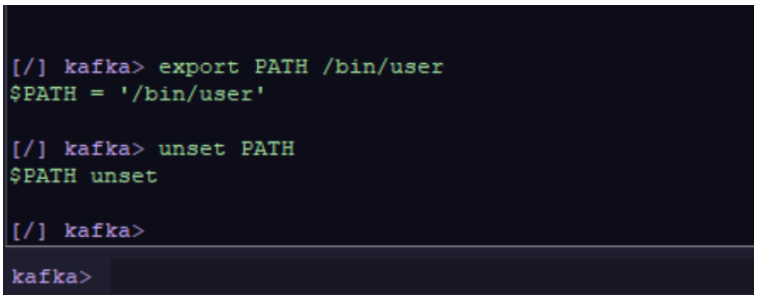
history Command

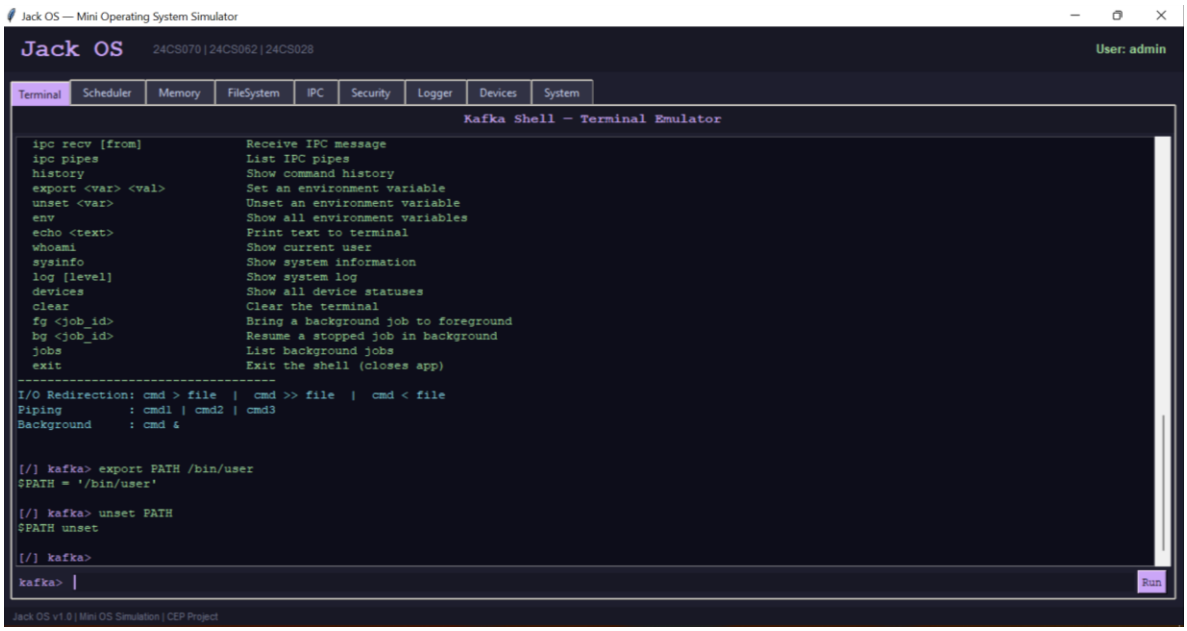
Displays a numbered list of all previously executed commands in the current session. Users can also navigate history using the Up and Down arrow keys in the input box.



export / unset Commands

export <VAR> <value> sets a named environment variable. unset <VAR> removes it. env lists all currently set variables. The $VAR syntax expands variables inside echo and other commands.





exit Command

Logs an "exit" event to the system logger, prints a farewell message, and cleanly exits the application window.

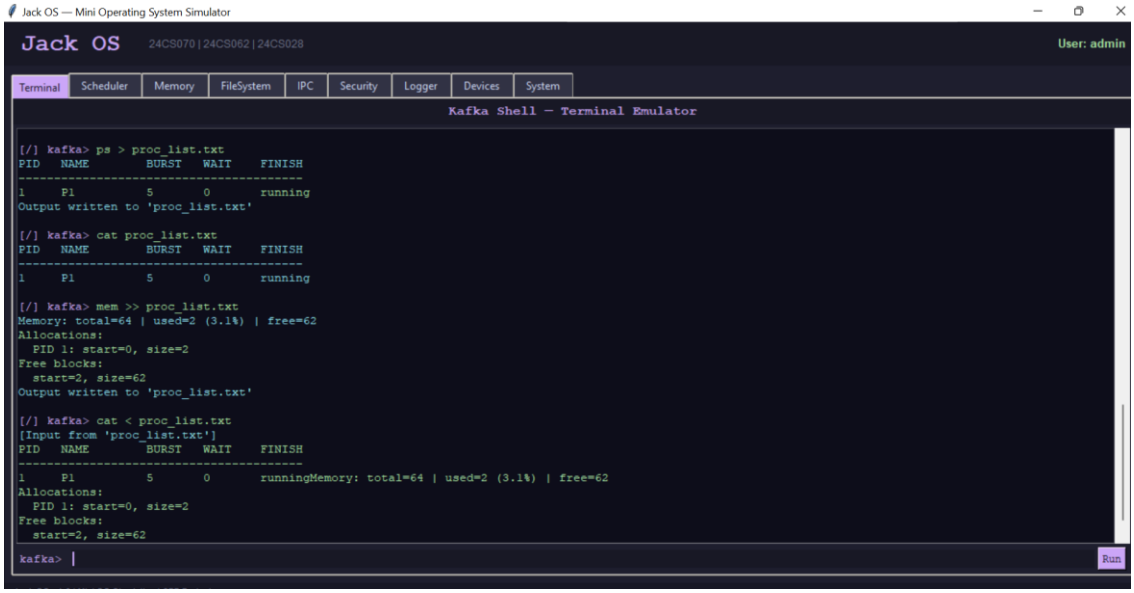
```
  exit                          Exit the shell (closes app)
-----------------------------------
I/O Redirection: cmd > file   |  cmd >> file   |  cmd < file
Piping         : cmd1 | cmd2 | cmd3
Background     : cmd &


[/] kafka>

kafka>
```

I/O Redirection

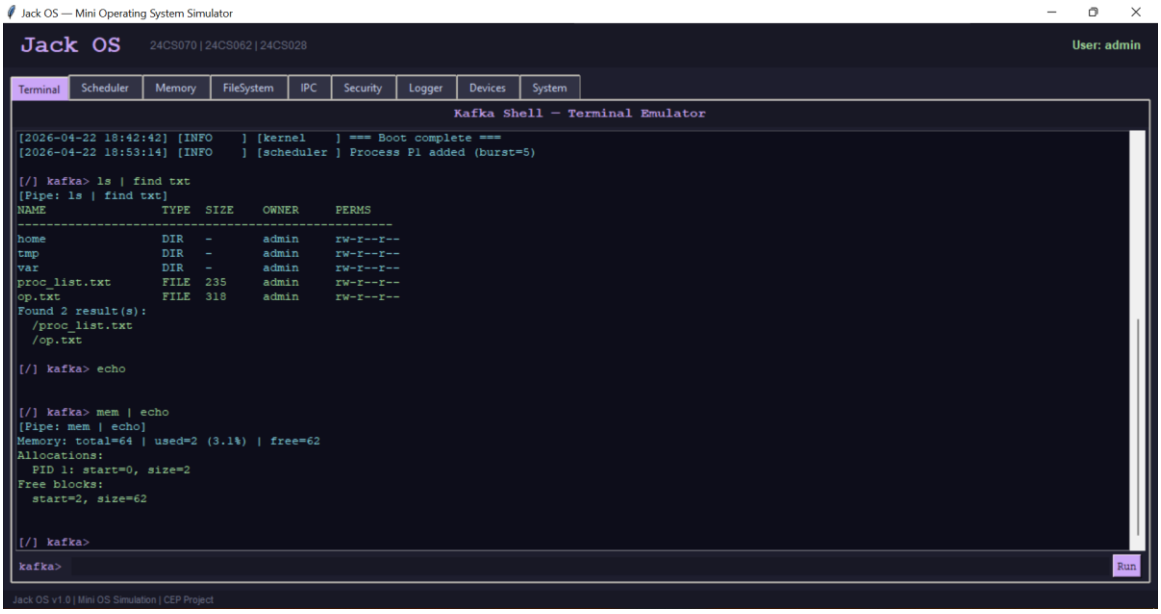
Redirects command output to a virtual file using > (overwrite) or >> (append). The < operator reads a file and displays its content as if it were input.



Piping Command

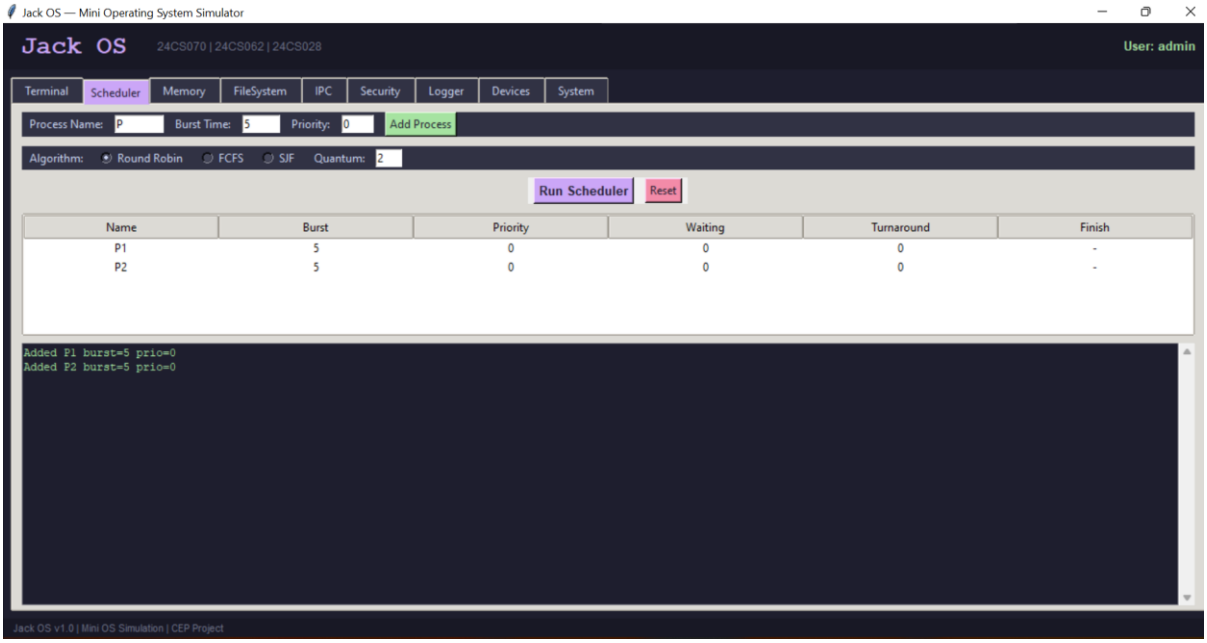
Uses the | operator to chain two or more commands together, feeding the output of one command as the input to the next. Example: ps | log info chains process listing into the logger.

```
[/] kafka> ps | log info
[Pipe: ps | log info]
PID   NAME        BURST   WAIT    FINISH
-----------------------------------------
1     P1          5       0       running
[2026-04-22 18:42:42] [INFO    ] [kernel    ] === Jack OS Kernel Booting ===
[2026-04-22 18:42:42] [INFO    ] [kernel    ] Version: 1.0.0
[2026-04-22 18:42:42] [INFO    ] [scheduler ] Initializing scheduler (Round Robin, quantum=2)
[2026-04-22 18:42:42] [INFO    ] [memory    ] Initializing memory manager (64 units)
[2026-04-22 18:42:42] [INFO    ] [fs        ] Initializing file system
[2026-04-22 18:42:42] [INFO    ] [ipc       ] Initializing IPC subsystem
[2026-04-22 18:42:42] [INFO    ] [security  ] Initializing security module
[2026-04-22 18:42:42] [INFO    ] [driver    ] Initializing device drivers
[2026-04-22 18:42:42] [INFO    ] [kernel    ] === Boot complete ===
[2026-04-22 18:53:14] [INFO    ] [scheduler ] Process P1 added (burst=5)
```



Scheduler Tab — GUI

Add processes with name, burst time, and priority. Select the algorithm (Round Robin, FCFS, or SJF) and set the time quantum for Round Robin. Click Run Scheduler to execute and view a Gantt-style execution log with per-process statistics and overall averages.



Memory Tab — GUI

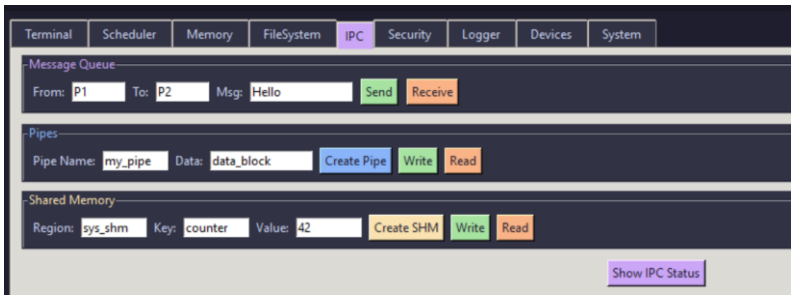
Allocate memory blocks using First Fit or Best Fit. A color-coded visual map shows each process in a distinct color. Deallocate any PID and click Compact to defragment. Summary shows total, used, free, and all free block positions.

FileSystem Tab — GUI

Create files and directories, write/read file content, change permissions (chmod), delete entries, search by name, and view the entire tree structure of the virtual filesystem.

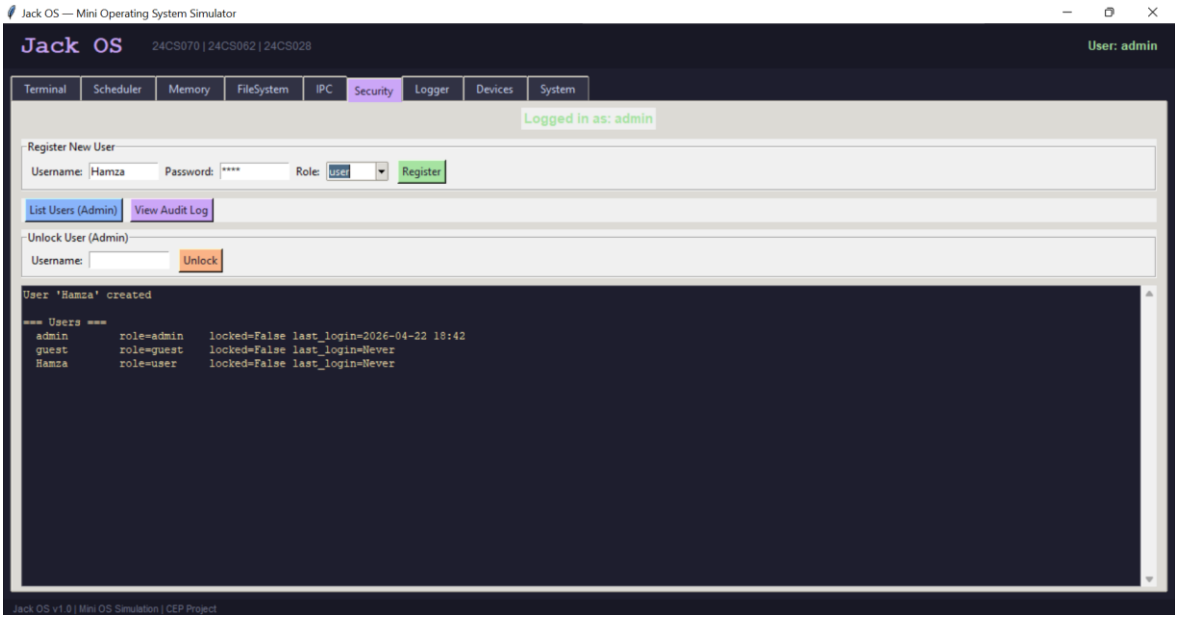
IPC Tab — GUI

Three panels: Message Queue (send/receive labeled messages), Pipes (create named pipes, write/read data), and Shared Memory (create named regions, store/retrieve key-value pairs). IPC Status button shows a full snapshot.



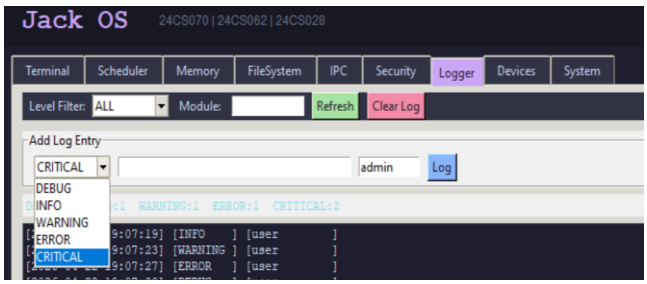
Security Tab — GUI

Register new users (admin only), view all user accounts with their roles and lock status, read the full timestamped audit log, and unlock accounts that were locked due to failed login attempts.



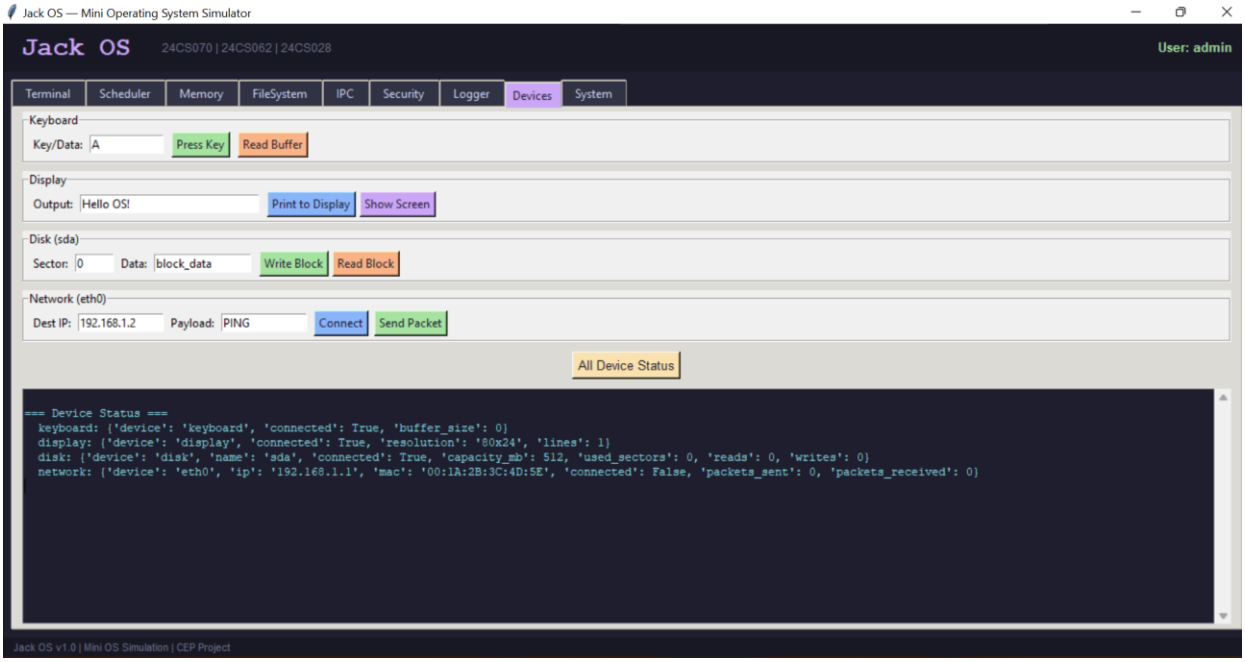
Logger Tab — GUI

Filter log entries by severity level (DEBUG/INFO/WARNING/ERROR/CRITICAL) and by module name. Add custom log entries manually. Refresh button reloads entries. Clear Log resets the log file and in-memory buffer.



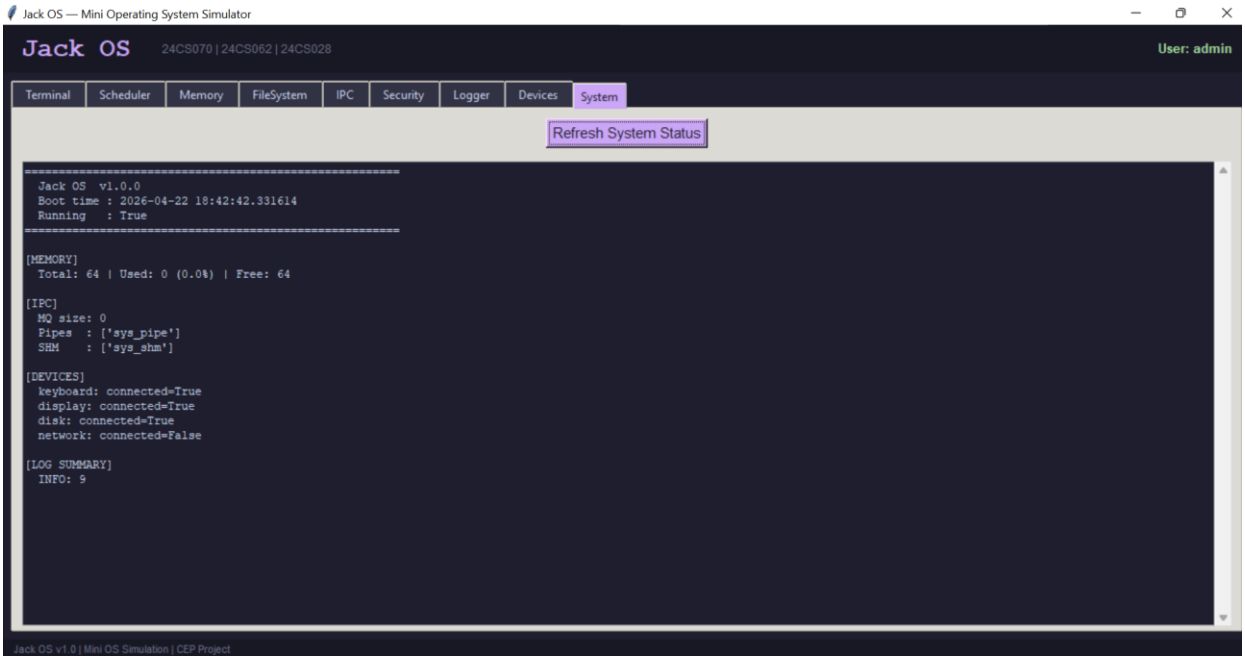
Devices Tab — GUI

Four device panels: Keyboard (press key events, read buffer), Display (print output, view screen), Disk (write/read sectors by number), and Network (connect, send packet to IP). All Device Status shows a full status report.



System Tab — GUI

Displays a live full-system snapshot: OS name and version, boot time, memory summary, IPC status (queue size, pipes, shared memory), device connection status, and log level counts.



6. User Documentation

Introduction

Welcome to Jack OS — Mini Operating System Simulator featuring the Kafka Shell, a lightweight custom shell designed for an enhanced command-line experience within a full graphical OS dashboard. This documentation provides comprehensive guidance on installation, functionality, and usage of the system.

Installation Instructions

Requirements:

- Python 3.8 or higher
- Tkinter — included with standard Python on Windows, macOS, and Linux
- No additional third-party packages required (standard library only)

Step 1 — Navigate to the project directory:

Jack OS — Project Structure

mini_os/ — Project Tree

```
mini_os/
│
├── kernel/
│  ├── __init__.py
│  └── kernel.py ← Central OS kernel controller
│
├── modules/
│  ├── __init__.py
│  ├── scheduler.py ← Round Robin + FCFS + SJF + stats
│  ├── memory.py ← First Fit + Best Fit + Compaction
│  ├── filesystem.py ← Virtual filesystem with permissions
│  ├── ipc.py ← Message Queue + Pipe + Shared Memory
│  ├── security.py ← Auth + roles + audit log
│  ├── logger.py ← Multi-level logging
│  └── driver.py ← Keyboard, Display, Disk, Network
│
├── gui/
│  ├── __init__.py
│  ├── app.py ← Main Tkinter GUI (9 tabs)
│  └── terminal.py ← Kafka Shell terminal emulator
│
├── main.py ← Entry point
├── log.txt ← Auto-generated system log
└── FINAL_REPORT.md ← Project documentation
```

```
cd mini_os
```

Step 2 — Run the application:

```
python main.py        (Windows)
python3 main.py       (Linux / macOS)
```

Step 3 — Login with default credentials:

Username	Password — Role
admin	admin123 — admin (full access to all features)
guest	guest — guest (read-only access)

Usage Instructions

- To execute a shell command, type it at the kafka> prompt in the Terminal tab and press Enter or click Run.
- For commands requiring arguments, follow the shown syntax — for example: kill <pid>, cd <directory>, export <VAR> <value>.
- Type help at any time to see all available commands with syntax and descriptions.
- Use Up / Down arrow keys to navigate through your command history.
- Press Tab to auto-complete partial command names.
- Switch between the 9 GUI tabs at any time to visually interact with any OS subsystem.

Shell Command Reference

Command	Syntax	Description
help	help	Display all available commands
cd	cd <dir> | cd .. | cd /	Change current directory
ls	ls [path]	List directory contents
pwd	pwd	Print working directory
mkdir	mkdir <name>	Create a directory
touch	touch <name>	Create an empty file
rm	rm <name>	Delete file or empty directory
cat	cat <name>	Print file contents
write	write <name> <text>	Write text to a file
chmod	chmod <name> <perm>	Change file permissions
find	find <name>	Search files by name
tree	tree	Show full filesystem tree
ps	ps	List all processes

spawn	spawn <name> <burst>	Create a new process
kill	kill <pid>	Kill process by PID
mem	mem	Show memory status
memalloc	memalloc <pid> <size>	Allocate memory block
memfree	memfree <pid>	Free memory for a PID
memcompact	memcompact	Defragment memory
ipc send	ipc send <recv> <msg>	Send IPC message
ipc recv	ipc recv [from]	Receive IPC message
ipc pipes	ipc pipes	List IPC pipes
ipc shm	ipc shm	Show shared memory
history	history	Show command history
export	export <VAR> <value>	Set environment variable
unset	unset <VAR>	Remove environment variable
env	env	List all env variables
echo	echo <text>	Print text (expands $VAR)
sysinfo	sysinfo	Full system snapshot
log	log [LEVEL]	Show system log
devices	devices	Show device status
fg	fg <job_id>	Bring job to foreground
bg	bg <job_id>	Resume job in background
jobs	jobs	List background jobs
exit	exit	Exit the shell

Functionalities

- Basic Commands: Execute standard OS operations (ls, pwd, echo, find, tree) directly in the Kafka Shell.

- Changing Directories: Use cd <directory> to navigate the virtual filesystem. cd .. moves up one level; cd / returns to root.
- Process Management: Spawn processes with spawn <name> <burst>, list them with ps, and terminate with kill <pid>.
- Memory Management: Allocate with memalloc <pid> <size>, free with memfree <pid>, and defragment with memcompact.
- File Operations: Create, read, write, delete, and search files; change permissions with chmod <name> <perm>.
- IPC: Send and receive named messages, write/read named pipes, and store key-value pairs in shared memory regions.
- Job Control: Run commands in background with &, manage active jobs with jobs, fg <id>, and bg <id>.
- History Management: The history command displays all previously executed commands. Up/Down arrows recall history in the input box.
- Environment Variables: export, unset, env, and inline $VAR expansion in commands and echo.
- I/O Redirection: Redirect output to a virtual file (>), append output (>>), or read file content as input (<).
- Piping: Combine commands using | to route output of one command as input to another.
- GUI Dashboard: 9 interactive tabs give visual access to all OS subsystems — Scheduler, Memory, FileSystem, IPC, Security, Logger, Devices, System.
- Security: Register users, manage roles (admin/user/guest), view timestamped audit logs, and unlock locked accounts.
- System Logging: 5-level centralized logger with per-module tagging, level filtering, and file export.

10. Conclusion

Jack OS — Mini Operating System Simulator with the Kafka Shell successfully demonstrates all core operating system concepts required by the Complex Engineering Problem framework. The project combines a practical command-line shell interface (Kafka Shell) with a full graphical OS dashboard, providing interactive simulation of every major OS subsystem.

All twelve bugs identified in the original partial implementation have been corrected: the missing Process import (NameError crash), absent scheduling statistics, missing FCFS and SJF algorithms, the off-by-one memory allocation bug, absent Best Fit and Compaction, missing filesystem CRUD operations, absent permission checking, incomplete IPC (no pipes or shared memory), plain-text password storage, single-level logging, trivial device driver simulation, and missing kernel coordination methods.

The Kafka Shell mirrors the complete feature set of the C-based Isekai Shell (functions.c) — including cd, ls, ps, kill, history, export, unset, exit, fg, bg, I/O redirection, and piping — reimplemented entirely in Python with integration into the OS kernel simulation. This project demonstrates the practical application of Operating Systems theory and provides a solid foundation for understanding how real OS kernels manage processes, memory, files, communication, and security.

RUBRICS

	Unacceptable	Poor	Acceptable	Adequate	Proficient	Score
R1 Problem/Requirement Identification	Problem not identified	Problem poorly identified	Problem is identified	Problem is defined adequately.	Problem is identified and analyzed in a well-defined manner.	
R2 Engineering knowledge (standards)	Can not apply engineering knowledge to the solution.	Has difficulty applying mathematics to the solution of complex engineering problems	Correctly applies basic sciences to the solution of complex engineering problems	Correctly applies engineering fundamentals to the solution of complex engineering problems	Correctly applies engineering specialization to the solution of complex engineering problems.	
R3 Efficiency of the solution	Solution does not meet requirements.	A difficult and inefficient solution.	A logical solution that is easy to follow but it is not the most efficient.	Solution is adequately efficient.	Solution is efficient, easy to understand, and maintain.	
R4 Technical Writing	The report is submitted but lacks solutions to major requirements.	The report submitted but not according to the requirements.	The requirements of report writing are not properly addressed.	Reports meets all prescribed requirements.	Reports meets all requirements, and it is prepared in original and corrective way to engage readers.	